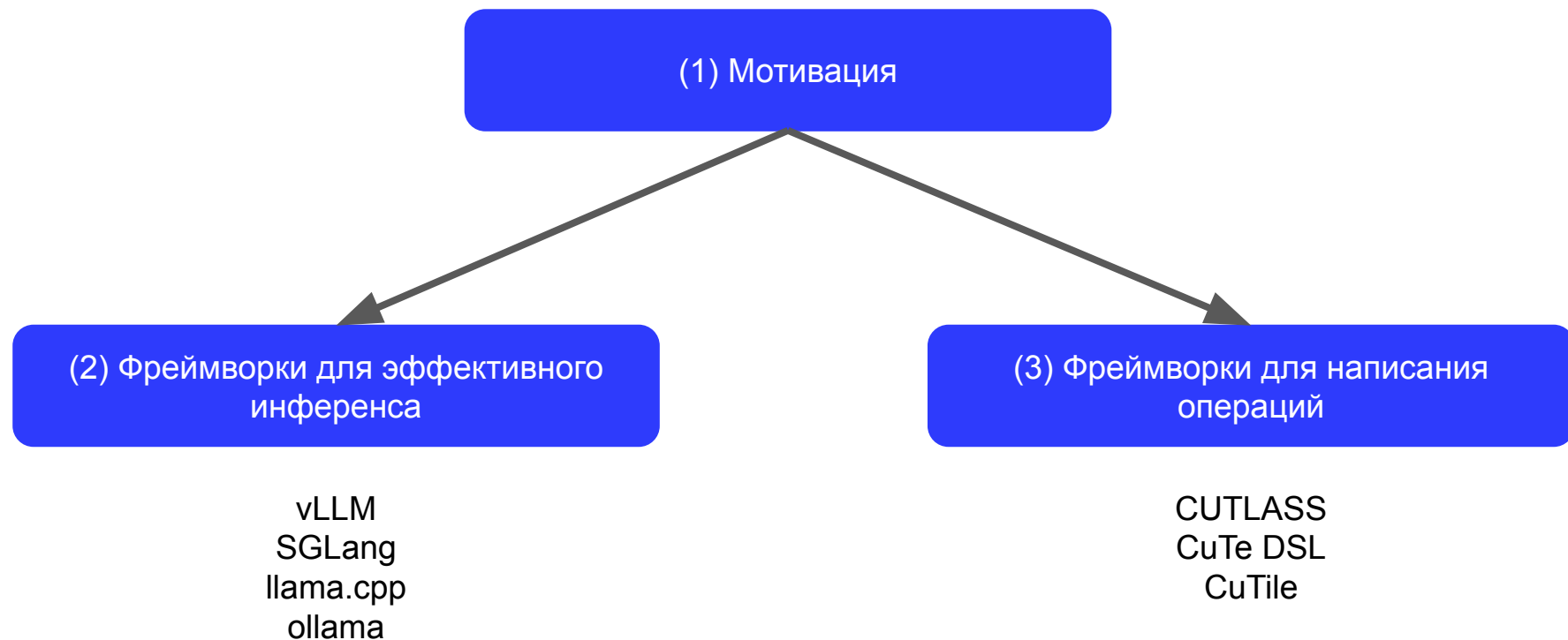


Библиотеки для эффективного инференса больших языковых моделей

Кузнецев Денис





Инференс больших языковых моделей требует существенных вычислительных затрат и операций по работе с памятью.

Наивные реализации в HuggingFace transformers / pure Torch неэффективно используют вычислительные ресурсы.



```
graph TD; A[Инференс больших языковых моделей требует существенных вычислительных затрат и операций по работе с памятью.] --> C[Нужны библиотеки с эффективными реализациями как отдельных операций, так и всего процесса инференса.]; B[Наивные реализации в HuggingFace transformers / pure Torch неэффективно используют вычислительные ресурсы.] --> C;
```

Нужны библиотеки с эффективными реализациями как отдельных операций, так и всего процесса инференса.



Серверный инференс



Локальный инференс



Один из первых
эффективных фреймворков
эффективного инференса.

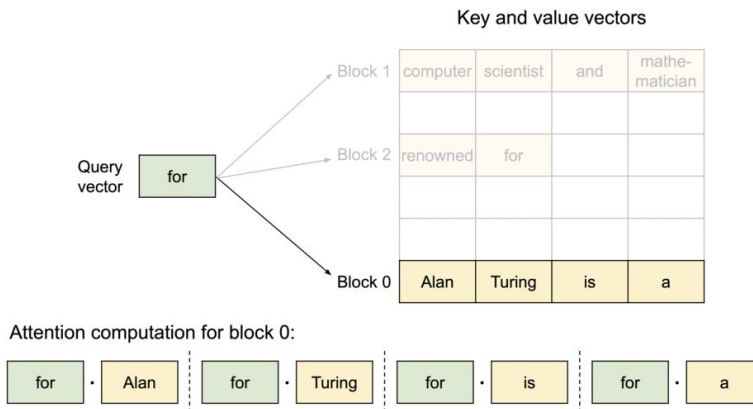


Ключевые особенности

- Эффективная работа с KV-кэшами при помощи **Paged Attention**
- Непрерывное батчевание запросов
- Быстрый инференс через CUDA/HIP графы
- Поддержка разных квантизаций GPTQ/AWQ, FP8, FP4
- Оптимизированные CUDA кернелы - интеграция с FlashAttention/FlashInfer
- **Спекулятивное декодирование**
- Chunked Prefill
- Работает как с LLM, так и с мультимодальными моделями

<https://docs.vllm.ai/en/latest/>

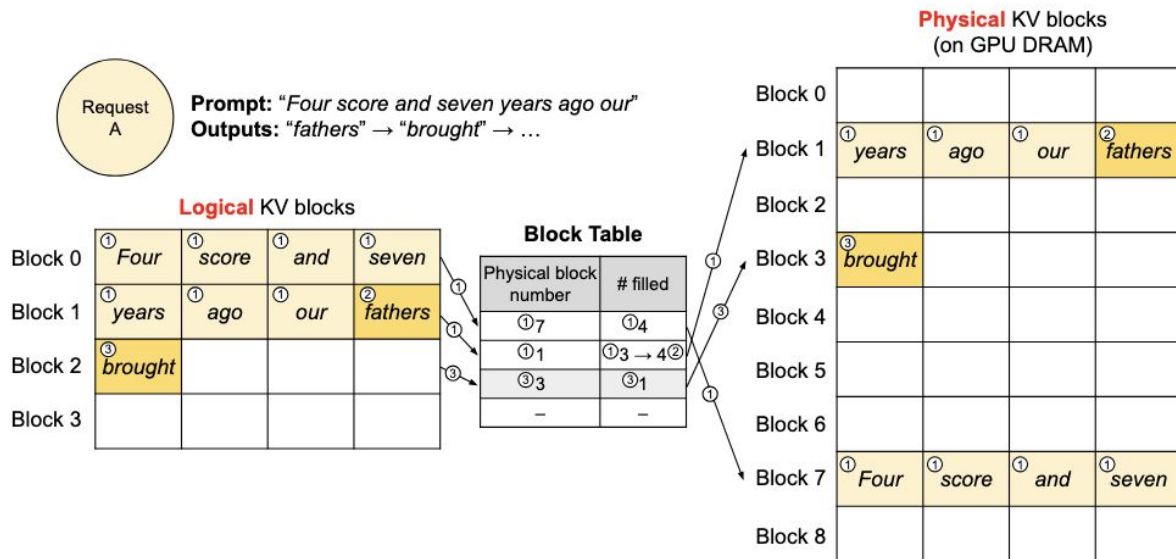
PagedAttention в vLLM — это способ хранить и читать KV-кеш (key/value для self-attention) не как один сплошной тензор на последовательность, а как набор фиксированных “страниц” (blocks) в GPU-памяти, похожий по идее на виртуальную память/пейджинг в ОС.



Paged Attention

- **KV кэш** режется на блоки фиксированного размера (по N токенов)
- **Глобальный пул блоков в GPU**: блоки выдаются и освобождаются динамически по мере генерации токенов разными запросами.
- **Таблица страниц (block table) на запрос**: для каждой последовательности хранится список ID блоков, которые ей принадлежат.
- При инференсе внутри attention происходит чтение блоков по таблице. Логически контекст **непрерывен**, физически — разбросан по **страницам**.

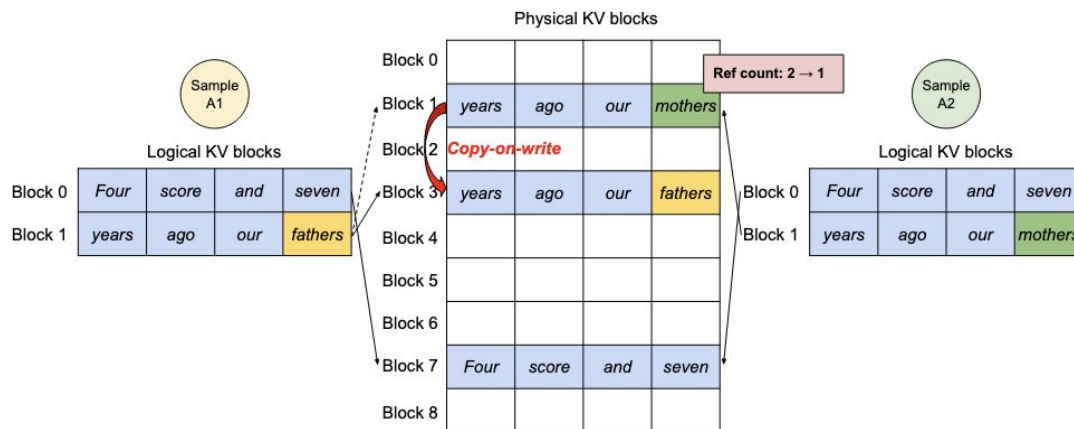
Paged Attention



Почему это выгодно?

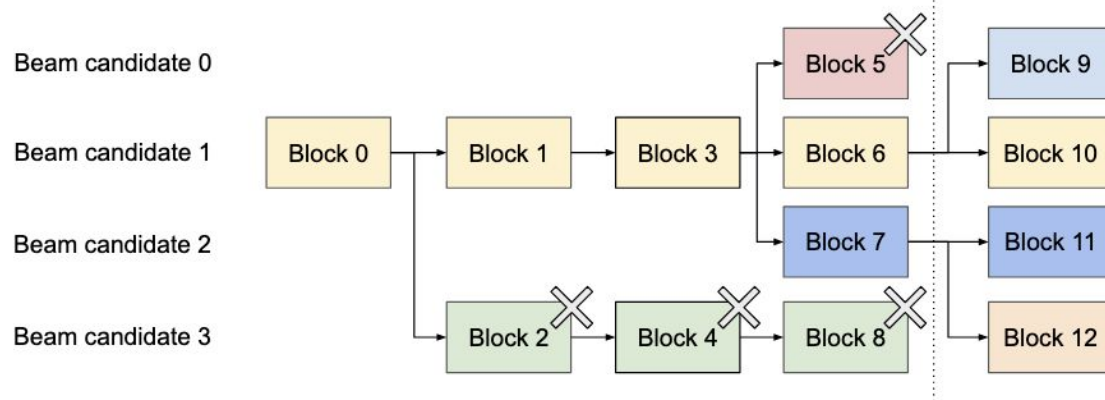
- Меньше **фрагментации** и перерасхода памяти (не нужно заранее резервировать большой непрерывный буфер под максимальную длину).
- Проще **добавлять токены**: как только блок заполняется, выделяется следующий.
- Эффективнее **батчевание** запросов с разной длиной и разной скоростью генерации (каждый растёт своим темпом, а память управляется блоками).
- Легче **переиспользовать префиксы** (например, общие промпты) — разные запросы могут ссылаться на одни и те же уже заполненные блоки (если включены соответствующие механизмы).

Paged Attention



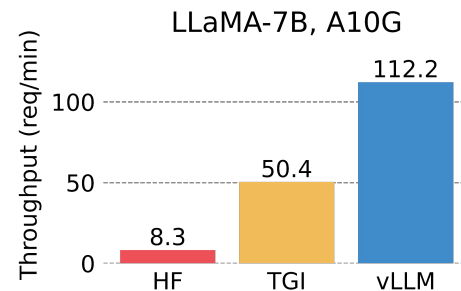
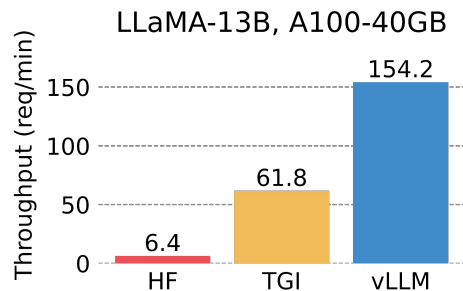
Параллельное сэмплирование

Paged Attention

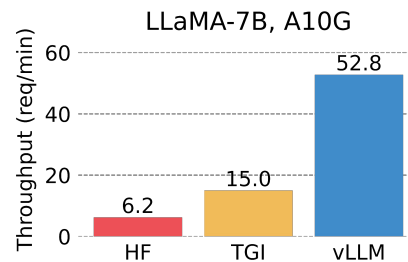
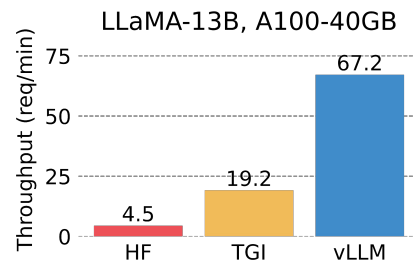


Beam Search v vLLM

Результаты



Сравнение пропускной способности при обработке одного запроса



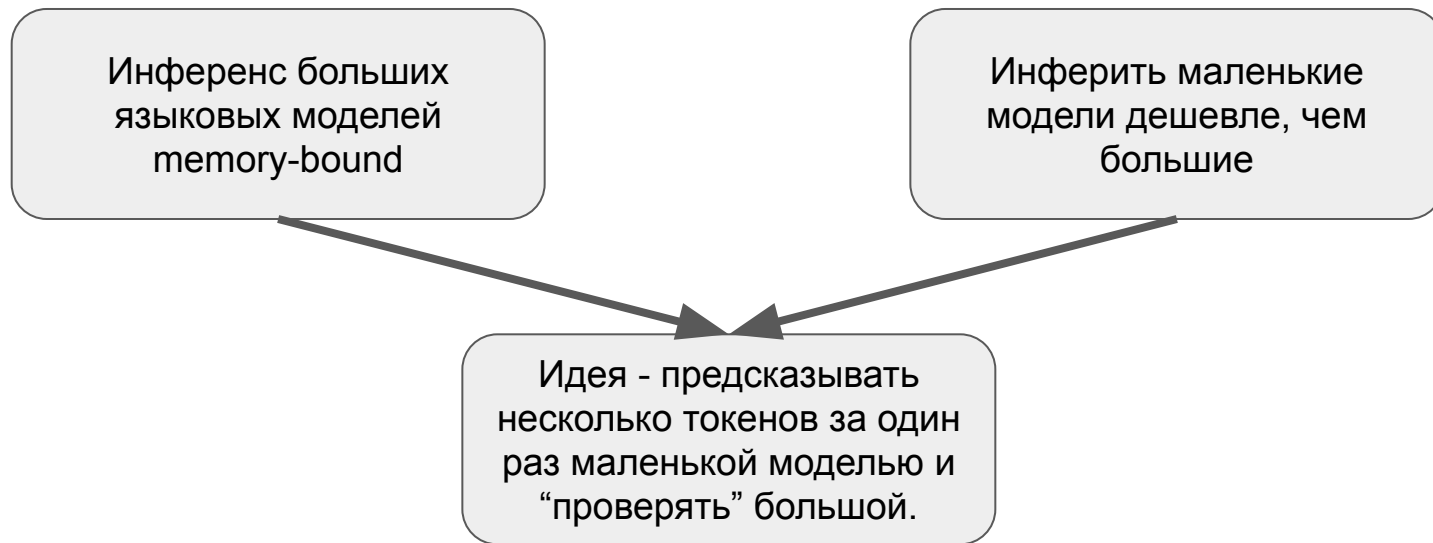
Сравнение пропускной способности при обработке трех запросов за раз

Результаты

Ускорение
до 24x раз против 🤗 transformers
до 3.5x раз против TGI*

*TGI - [HuggingFace Text Generation Inference \(TGI\)](#)

Спекулятивное декодирование



<https://arxiv.org/abs/2211.17192>

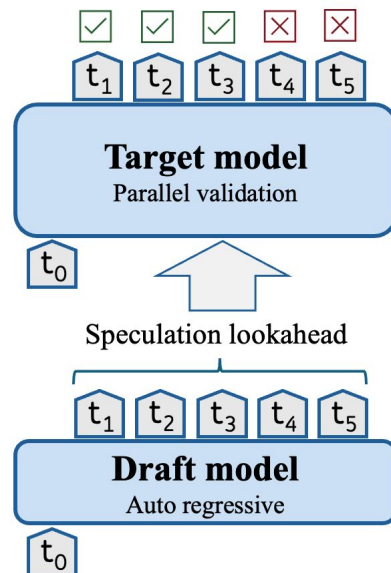
Спекулятивное декодирование

Можно валидировать несколько токенов за один forward pass с почти той же стоимостью прямого прохода, что и один токен.

“Дешевая” модель, сэмплирующая токены, называется драфт-моделью.

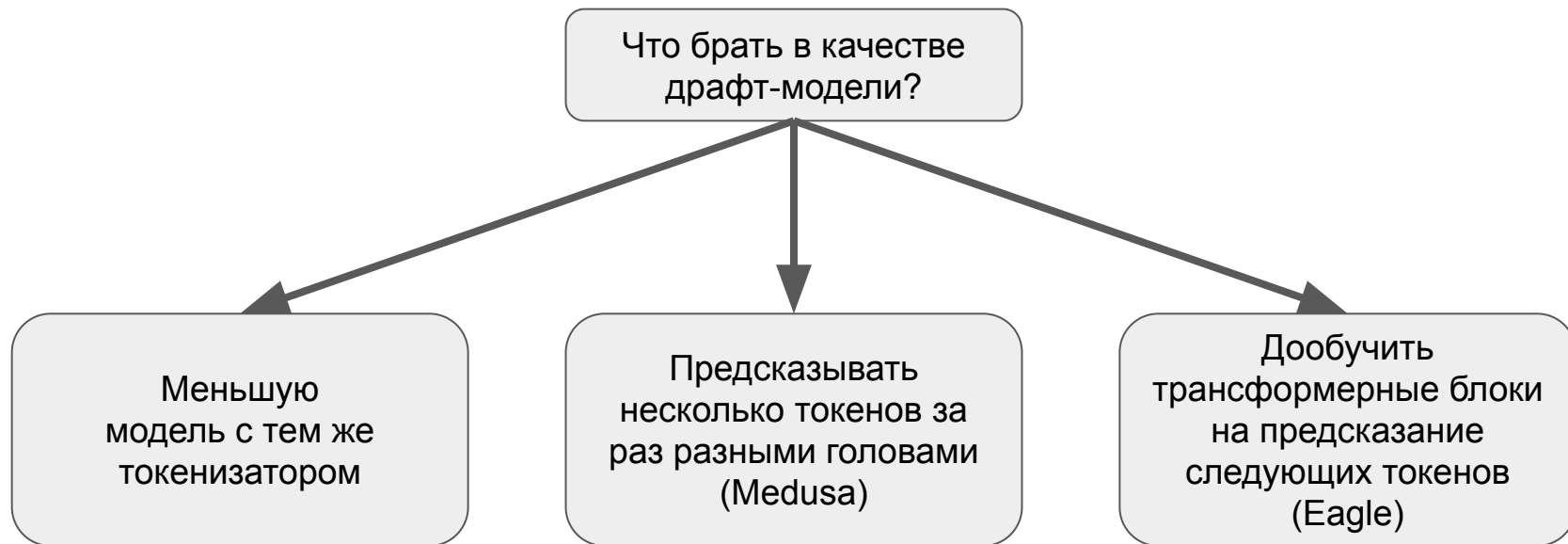
“Дорогая” модель, проверяющая токены, называется таргет-моделью.

На выходе имеем генерацию как у таргет модели, но быстрее.



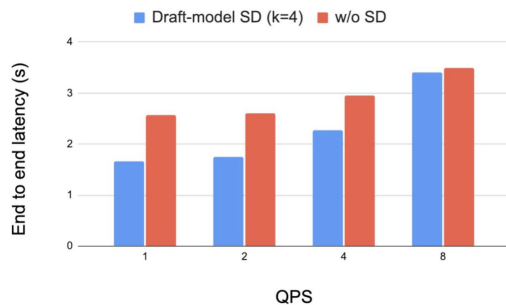
https://huggingface.co/blog/dynamic_speculation_lookahead

Спекулятивное декодирование



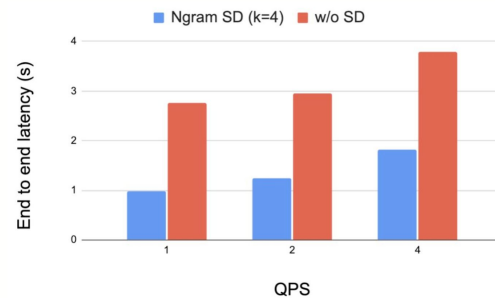
Спекулятивное декодирование

Llama3-70B on ShareGPT @ 4xH100



Up to 1.5x
Speedup at
QPS=1

Llama3-70B on CNN Dailymail @ 4xH100

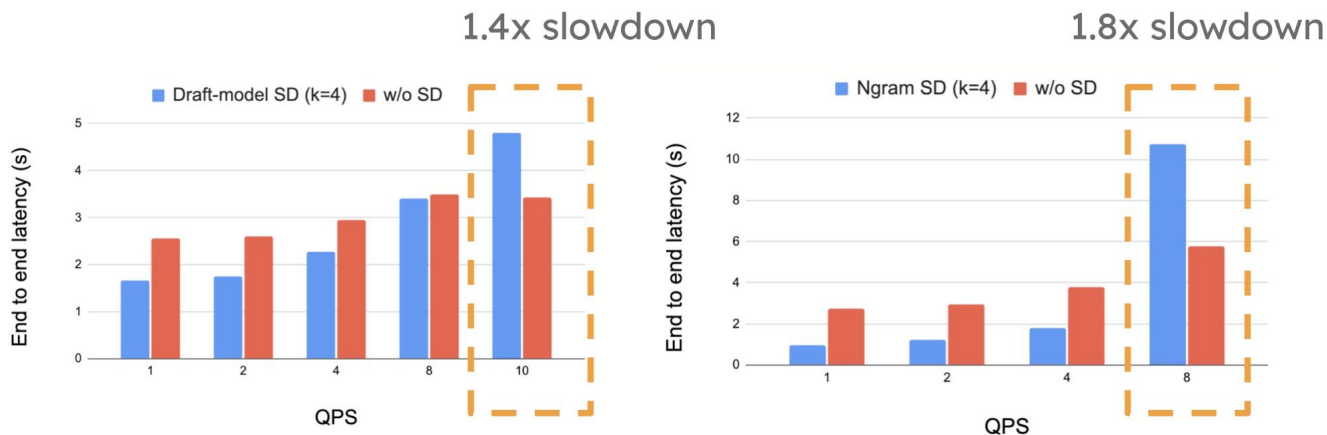


Up to 2.8x
Speedup at
QPS=1

Ускорение генерации до 3х раз

<https://blog.vllm.ai/2024/10/17/spec-decode.html>

Спекулятивное декодирование



Если у вас compute-bound сценарий, спекулятивное декодирование может замедлить генерацию.

<https://blog.vllm.ai/2024/10/17/spec-decode.html>

Спекулятивное декодирование

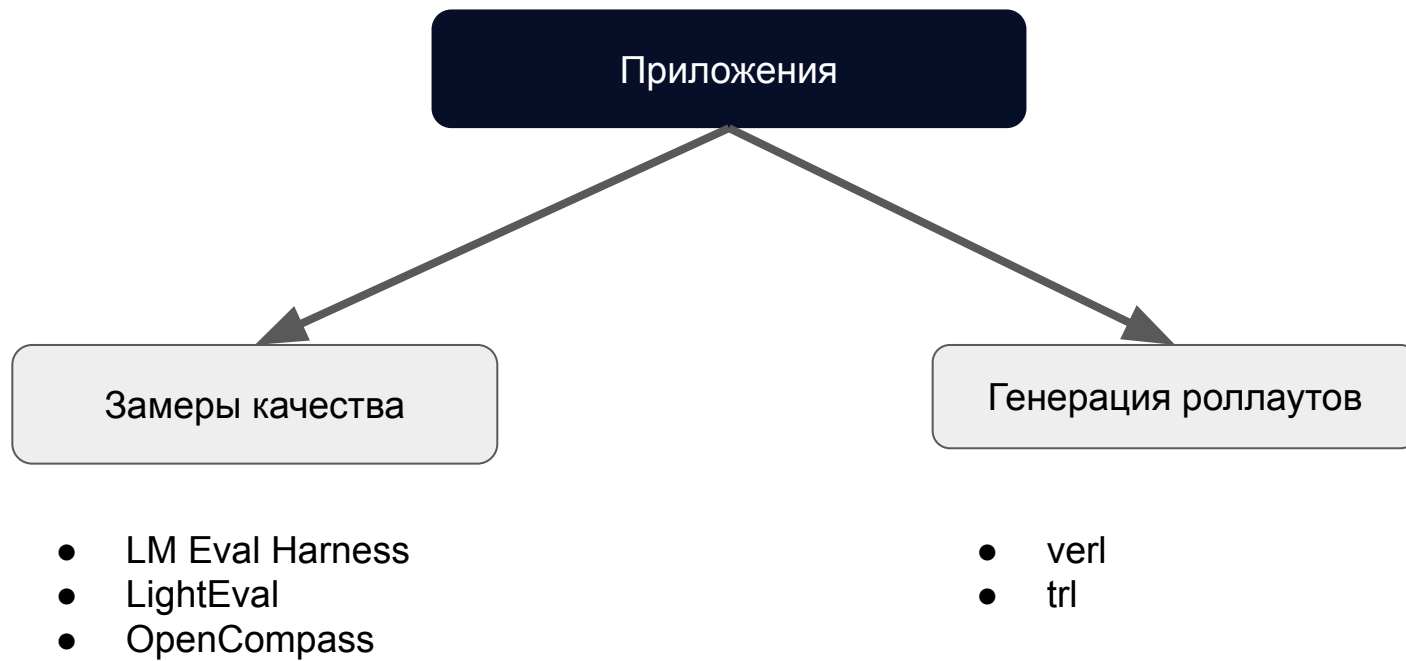
```
from vllm import LLM

llm = LLM(
    model="facebook/opt-6.7b",
    speculative_model="facebook/opt-125m",
    num_speculative_tokens=5,
)
outputs = llm.generate("The future of AI is")

for output in outputs:
    print(f"Prompt: {output.prompt!r}, Generated text: {output.outputs[0].text!r}")
```

Пример использования в vLLM

<https://blog.vllm.ai/2024/10/17/spec-decode.html>





Конкурентный фреймворк
эффективного инференса.

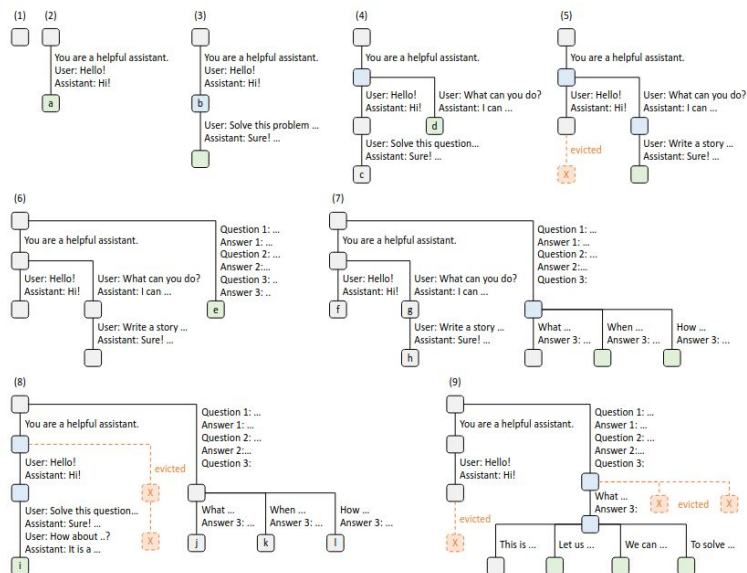


Ключевые особенности

- Эффективная работа с KV-кэшами при помощи **Radix Attention**.
- Кэширование префиксов.
- Префилл-декодирование дисаггрегация.
- Спекулятивное декодирование.
- Непрерывное батчевание.
- Tensor/Pipeline/Expert/Data parallelism.
- Квантизация - INT4, INT8, FP4, FP8.
- Multi-LoRA батчевание.

<https://docs.vllm.ai/en/latest/>

Radix Attention в SGLang — это способ организовать **KV-кэш (key/value)** так, чтобы **много запросов могли эффективно делить общие префиксы** (одинаковые начала промптов) и не пересчитывать/не хранить их заново.



Принцип работы Radix Attention

- **Радикс-дерево (trie) по токенам.** Все уже посчитанные последовательности токенов хранятся в виде дерева.
- **KV для каждого узла/сегмента.** В узлах дерева лежат соответствующие **K/V тензоры** (часто блоками), полученные при прогоне модели по этому префиксу.
- **Поиск “самого длинного совпадающего префикса”.** При приходе запроса система токенизирует префикс, проходит по radix-дереву и находит **max common prefix**.
 - Совпавшая часть: KV берётся из кэша.
 - Остаток: догоняется вычислением, затем добавляется в дерево.
- **Динамическое батчирование без потери шаринга.** Разные запросы могут находиться на разных глубинах дерева; при генерации следующие токены считаются уже “поверх” разделяемого контекста.
- **Управление памятью (evict/TTL/LRU-подобное).** Когда память KV переполняется, дерево позволяет **выкидывать целые ветки/листья**, минимально ломая полезные общие префиксы.

Пример использования

```

dimensions = ["Clarity", "Originality", "Evidence"]

@function
def multi_dimensional_judge(s, path, essay):
    s += system("Evaluate an essay about an image.")
    s += user(image(path) + "Essay:" + essay)
    s += assistant("Sure!")

    # Return directly if it is not related
    s += user("Is the essay related to the image?")
    s += assistant(select("related", choices=["yes", "no"]))
    if s["related"] == "no": return

    # Judge multiple dimensions in parallel
    forks = s.fork(len(dimensions))
    for f, dim in zip(forks, dimensions):
        f += user("Evaluate based on the following dimension:" +
            dim + ". End your judgment with the word 'END'")
        f += assistant("Judgment:" + gen("judgment", stop="END"))

    # Merge the judgments
    judgment = "\n".join(f["judgment"] for f in forks)

    # Generate a summary and a grade. Return in the JSON format.
    s += user("Provide the judgment, summary, and a letter grade")
    s += assistant(judgment + "In summary," + gen("summary", stop=".")
        + "The grade of it is" + gen("grade"))

    schema = r'{"summary": "[\w\d\s]+\.", "grade": "[ABCD][+]?"}'
    s += user("Return in the JSON format.")
    s += assistant(gen("output", regex=schema))

state = multi_dimensional_judge.run(...)
print(state["output"])

```

Handle chat template and multi-modal inputs

Select an option with the highest probability

Fetch result; Use Python control flow

Runtime optimization: KV Cache Reuse (Sec. 3)

Multiple generation calls run in parallel

Fetch generation results

Runtime optimization: API speculative execution (Sec. 5)

Runtime optimization: fast constrained decoding (Sec. 4)

Run an SGLang program

<https://arxiv.org/abs/2312.07104>

Результаты

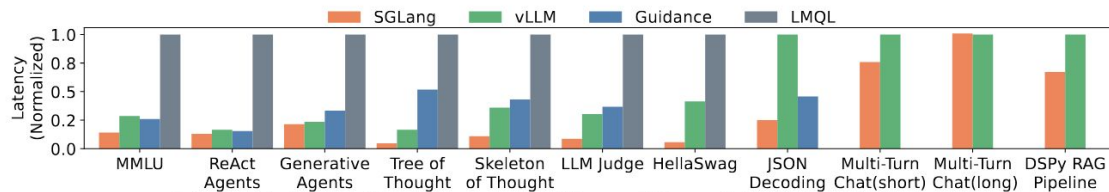


Figure 6: Normalized latency on Llama-7B models. Lower is better.

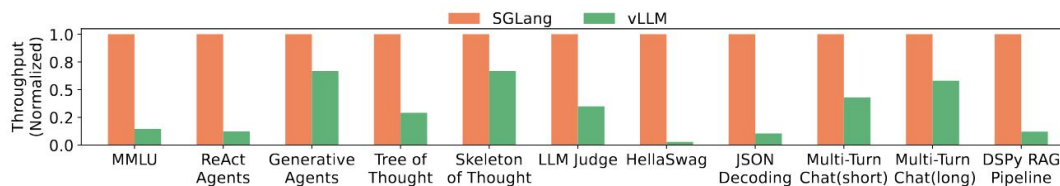


Figure 7: Normalized throughput on Mixtral-8x7B models with tensor parallelism. Higher is better.

Утверждается, что даже быстрее чем vLLM



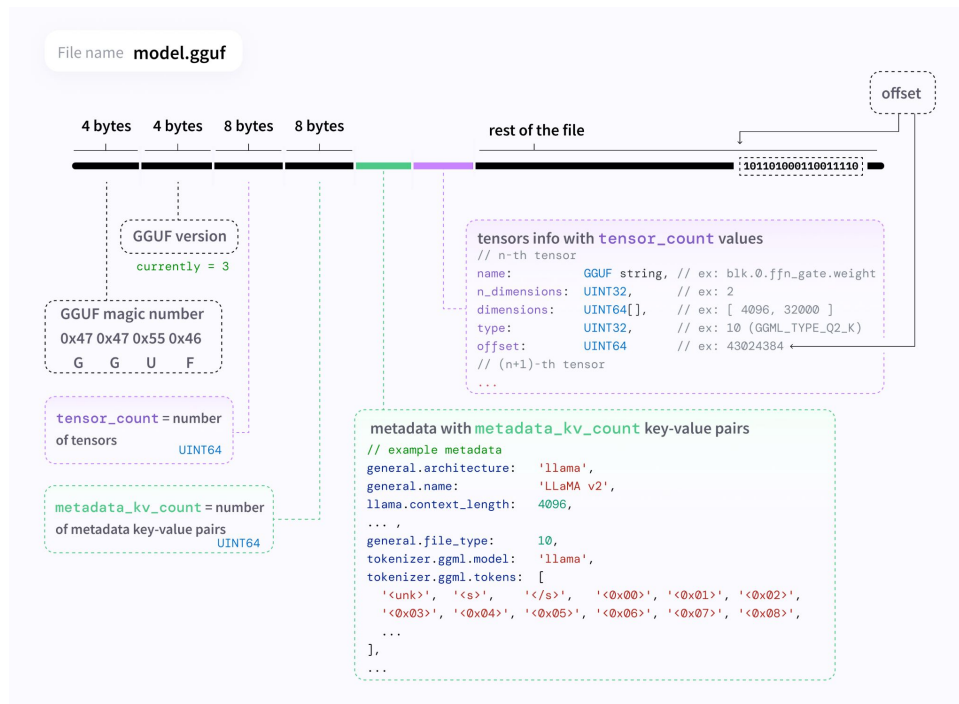


Фреймворк, написанный на
C/C++, для локального и
серверного инференса.

Ключевые особенности

- Поддержка Apple Silicon
- Поддержка AVX, AVX2, AVX512 and AMX для x86
- 1.5-, 2-, 3-, 4-, 5-, 6-, и 8- битная квантизация для ускорения инференса и снижения потребления памяти.
- Специальные CUDA кернелы для NVIDIA и AMD GPU.
- Гибридный CPU+GPU инференс.
- Поддержка мультимодальных моделей.

GGUF форматы



GGUF форматы

- **Q8_0** — “почти безболезненно”, часто хороший баланс качества/скорости, но файл крупнее.
- **Q6_K** — высокий уровень качества, меньше, чем Q8.
- **Q5_K_M / Q5_K_S** — популярный баланс (M обычно чуть лучше/тяжелее, S чуть легче).
- **Q4_K_M / Q4_K_S** — самый массовый “разумный минимум” для локального запуска; ощутимая экономия памяти.
- **Q3_K_M / Q3_K_S, Q2_K** — ещё компактнее, но деградация качества заметнее (особенно на сложных задачах/длинных ответах).

Суффиксы:

- **K** — семейство “k-quant” (современные схемы квантования в llama.cpp, обычно лучше старых).
- **_M / _S** — варианты компромисса (условно *Medium/Small* по качеству/размеру).



ollama - это удобная
оболочка/платформа для
локального запуска LLM.
Использует под капотом
llama.cpp.

Ключевые особенности

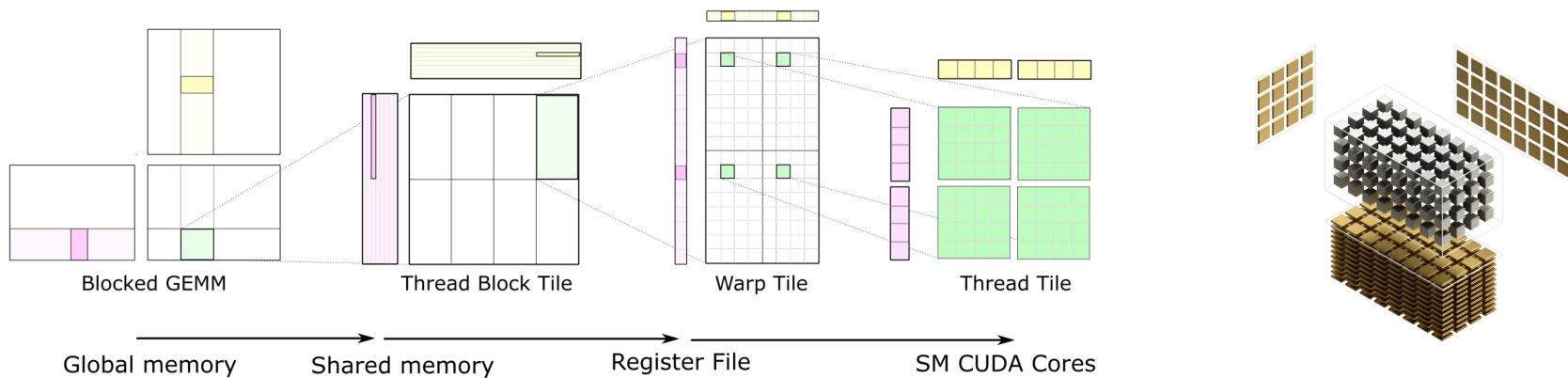
- Простая установка и запуск.
- Каталог моделей и управление ими.
- Modelfile (рецепты сборки модели).
- Фокус на практику, а не на “зоопарк настроек”. По сравнению с чистым llama.cpp меньше низкоуровневого контроля, зато быстрее старт и меньше шансов “сломать” конфиг.

ollama vs llama.cpp

Критерий	llama.cpp	Ollama
Цель	Эффективный локальный инференс как библиотека/CLI	Удобный запуск моделей “как сервис”
Установка/старт	Нужно самому выбрать модель (часто GGUF), параметры, команду	<code>ollama run ...</code> — максимально просто
Управление моделями	Вручную (скачал файл/путь/версии сам)	Встроенный каталог, <code>pull</code> , кеш, версии
Форматы/совместимость	В первую очередь GGUF (экосистема llama.cpp)	Использует свои “пакеты” моделей; под капотом может быть GGUF/движок типа llama.cpp
Контроль над инференсом	Очень высокий: квантование, offload, параметры, сборка, патчи	Средний: основные параметры есть, но часть деталей скрыта/ограничена
Производительность	Часто максимум возможного при правильной настройке	Обычно близко, но зависит от того, как Ollama упаковал/сконфигурировал модель и бэкэнд
API/сервер	Есть пример-сервер, но “продуктивности” меньше	Встроенный удобный локальный HTTP API + streaming
Встраивание в приложения	Отлично: библиотека, тонкая интеграция	Обычно через API (проще), но меньше контроля внутри процесса
Воспроизводимость (настройки “рецептом”)	Скрипты/флаги/конфиги — делаете сами	Modelfile (рецепты): системный промпт, параметры, сборка производных моделей
Кому подходит	Энтузиастам, разработчикам движка, тем кому нужна оптимизация/кастом	Разработчикам приложений, быстрым прототипам, локальным ассистентам

Рассмотренные выше
решения были про ускорение
системы в целом.

Как ускорять отдельные
операции?



CUTLASS (CUDA Templates for Linear Algebra Subroutines and Solvers) — это открытый C++-шаблонный фреймворк от NVIDIA для построения высокопроизводительных примитивов линейной алгебры на GPU (в первую очередь GEMM и связанные операции), которые лежат в основе многих DL/HPC-нагрузок.

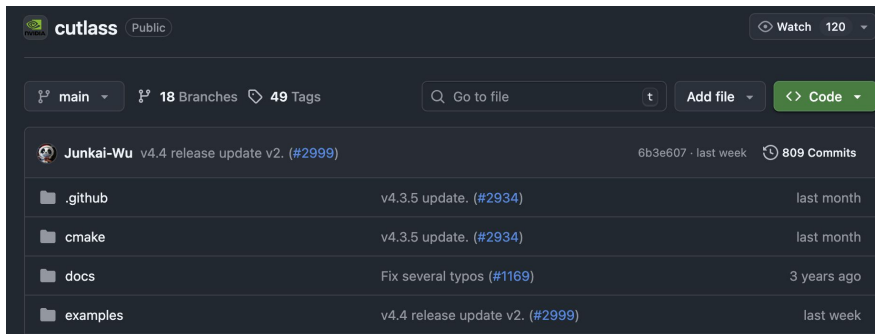
Что это такое и зачем нужно?

- **Библиотека/фреймворк “строительных блоков”**: позволяет собирать ядра матричного умножения и родственных операций из параметризуемых компонентов.
- **Ориентация на производительность**: использует особенности архитектур NVIDIA (иерархия памяти, warp-level примитивы, Tensor Cores через WMMA/MMA и т. п.).
- **Цель**: дать разработчикам уровень ниже cuBLAS/cuDNN — возможность **настраивать** вычислительные ядра под конкретные размеры, типы данных, раскладки и фьюзинг.

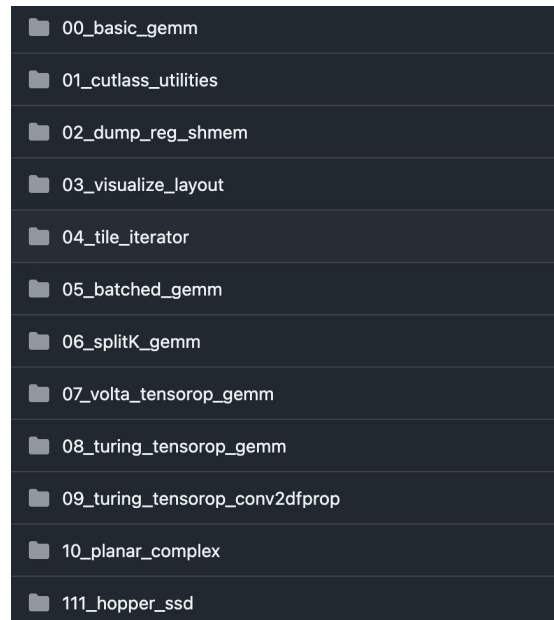
Основные возможности

- **GEMM и варианты:** классическое матричное умножение, batched/strided batched, разные раскладки (row/column major), эпилоги (bias, activation, scaling).
- **Поддержка Tensor Cores:** FP16/BF16/TF32/INT8 и др. режимы (зависит от GPU) с MMA-инструкциями.
- **Слияние в эпилоге:** часто можно объединять в один kernel операции после GEMM (например, add+bias+ReLU), уменьшая трафик в память.
- **Шаблонная параметризация:** tile-размеры, стратегия загрузки в shared memory, организация warp-ов, типы данных, выравнивания, pipeline stages.
- **Примеры и “reference” реализации:** помогает как для обучения, так и как база для кастомных ядер.

ОСНОВНЫЕ ВОЗМОЖНОСТИ



В папке **examples** можно найти ~100 примеров операций на CUTLASS - от простых до сложных, от Volta до Blackwell архитектур.



CuTe (CUDA Tensor Expression) — это Python-based domain-specific language (DSL) и сопутствующий инструментарий от NVIDIA, для **высокоуровневого описания тензорных вычислений** и их автоматической генерации под GPU (NVIDIA CUDA).

https://docs.nvidia.com/cutlass/latest/media/docs/pythonDSL/cute_dsl.html

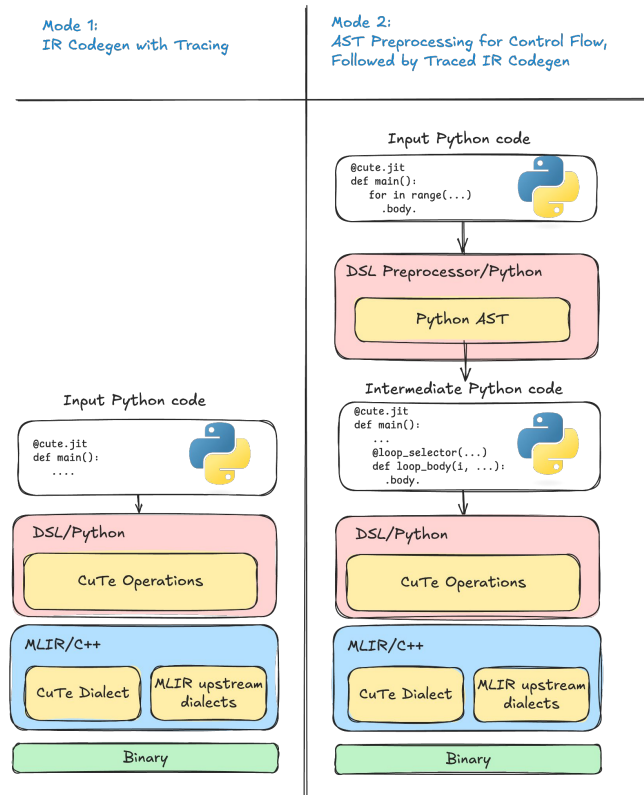
Для чего оно?

- Позволяет описывать тензорные операции (например, матричное умножение, свёртки, редукции) в компактной, математически наглядной форме.
- На основе этих описаний автоматически генерирует оптимизированный CUDA-код, в том числе с использованием **Tensor Cores** и других аппаратных особенностей GPU.
- Ориентирован на **ускорение разработки** кастомных высокопроизводительных ядер без необходимости вручную писать низкоуровневый CUDA.

Ключевые особенности

- **Синтаксис, близкий к математической нотации:** описание операций через индексы и тензорные выражения (наподобие Einsum).
- **Автогенерация кода:** из высокоуровневого описания получается CUDA-kernel с оптимизацией под целевую архитектуру (SM version, Tensor Cores и т. п.).
- **Поддержка разных типов данных и режимов Tensor Cores:** FP16, BF16, TF32, INT8 и др.
- **Интеграция с CUDA:** сгенерированный код компилируется штатными средствами (nvcc) и может встраиваться в существующие CUDA-приложения.
- **Гибкость:** позволяет задавать разбиение на плитки, стратегии загрузки в shared memory, pipeline и пр. через параметры DSL.

Этапы преобразования кода в исполняемый файл



`pip install nvidia-cutlass-dsl`



```
import cutlass
import cutlass.cute as cute

@cute.kernel
def kernel():
    tidx, _, _ = cutlass.nvvm.thread_idx()
    if tidx == 0:
        cute.print_("Hello world")

@cute.jit
def host():
    kernel().launch(
        grid=(1, 1, 1), block=(32, 1, 1)))

host()
```



`python3 hello_world.py`

Пример первой программы на CuTe

Отличия CuTe DSL от CUTLASS

Уровень абстракции

- **CuTe** — *высокоуровневый DSL*: вы пишете тензорное выражение (например, `C[i, j] += A[i, k] * B[k, j]`), а система сама строит ядро.
- **CUTLASS** — *низкоуровневый фреймворк*: вы компонуете ядра из готовых шаблонных компонентов (tile iterators, MMA ops, epilogues), контролируя детали реализации.

Гибкость vs скорость разработки

- **CuTe**: быстрее прототипировать и менять операции, но меньше контроля над низкоуровневыми оптимизациями.
- **CUTLASS**: больше контроля и возможностей тонкой настройки (например, кастомные pipeline, специфичные схемы загрузки), но выше порог входа и трудозатраты.

Генерация кода

- **CuTe**: автоматически генерирует CUDA-код из DSL-описания.
- **CUTLASS**: не генерирует код «из ничего» — вы пишете C++ с использованием шаблонных классов и примитивов CUTLASS.

Отличия CuTe DSL от Triton

CuTe - библиотека шаблонов. В ней явно задается отображение данных на индексы потоков/варпов/блоков.

Явный контроль над shared memory.

Компиляция через nvcc.

Явное задание layout-ов.



Triton - блочная модель программирования. Нет явного контроля над потоками и варпами.

Нет контроля над shared memory.

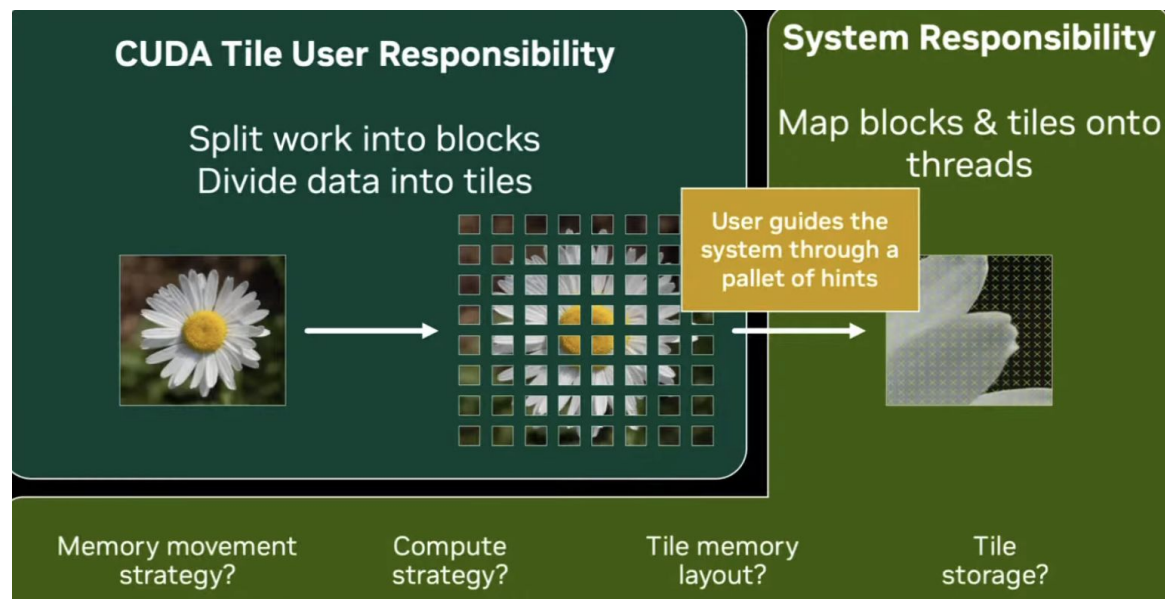
Just-in-time компиляция.

Компилятор сам пытается оптимизировать layout-ы.



CUDA Tile — это библиотека
для определения/написания tiled GPU ядер
с использованием стандартного питоновского синтаксиса.

<https://docs.nvidia.com/cuda/cutile-python/>



Пользователь задает разбиение данных на блоки и тайлы.

Система определяет low-level вычисления.

<https://docs.nvidia.com/cuda/cutile-python/>

```
import cuda.tile as ct
import cupy

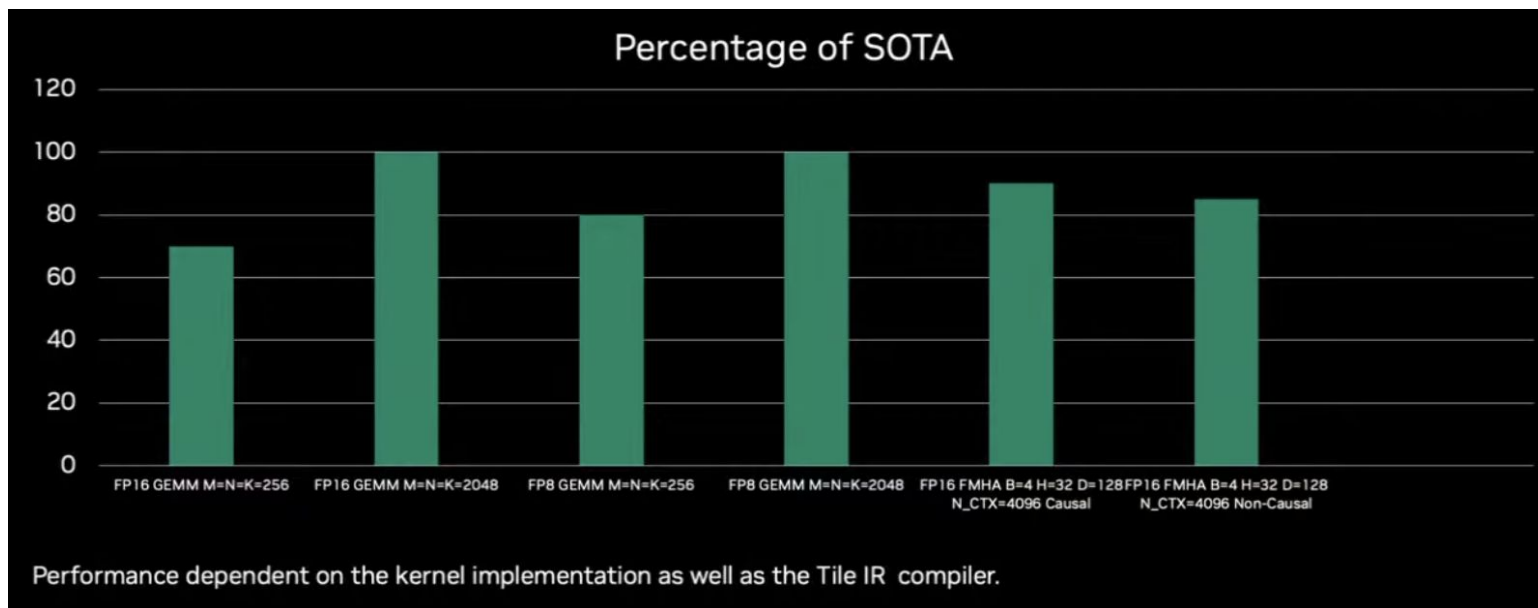
TILE_SIZE = 16

# cuTile kernel for adding two dense vectors. It runs in parallel on the GPU.
@ct.kernel
def vector_add_kernel(a, b, result):
    block_id = ct.bid(0)
    a_tile = ct.load(a, index=(block_id,), shape=(TILE_SIZE,))
    b_tile = ct.load(b, index=(block_id,), shape=(TILE_SIZE,))
    result_tile = a_tile + b_tile
    ct.store(result, index=(block_id,), tile=result_tile)

# Host-side function that launches the above kernel.
def vector_add(a: cupy.ndarray, b: cupy.ndarray, result: cupy.ndarray):
    assert a.shape == b.shape == result.shape
    grid = (ct.cdiv(a.shape[0], TILE_SIZE), 1, 1)
    ct.launch(cupy.cuda.get_current_stream(), grid, vector_add_kernel, (a, b, result))
```

Пример кернела на cuTile

https://www.youtube.com/watch?v=_b4l4rKpsGA&t=4432s



Производительность в сравнении с SOTA на B200

https://www.youtube.com/watch?v=_b4l4rKpsGA&t=4432s

Спасибо за

$$\text{softmax}\left(\frac{\begin{matrix} \text{Q} \\ \begin{matrix} \begin{matrix} \square & \square & \square \\ \square & \square & \square \end{matrix} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{matrix} \square & \square \\ \square & \square \\ \square & \square \end{matrix} \end{matrix}}{\sqrt{d_k}}\right)$$